



Core Concepts

Metrics

Available Metrics

Retrieval Augmented Generation

Context Precision

[Copy page](#)

Context Precision is a metric that evaluates the retriever's ability to rank relevant chunks higher than irrelevant ones for a given query in the retrieved context. Specifically, it assesses the degree to which relevant chunks in the retrieved context are placed at the top of the ranking.

It is calculated as the mean of the precision@k for each chunk in the context. Precision@k is the ratio of the number of relevant chunks at rank k to the total number of chunks at rank k.

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{(\text{true positives@k} + \text{false positives@k})}$$

Where K is the total number of chunks in `retrieved_contexts` and $v_k \in \{0, 1\}$ is the relevance indicator at rank k .

Examples

Context Precision

[latest](#)

The `ContextPrecision` metric evaluates whether retrieved contexts are useful for answering a question by comparing each context against a reference answer. Use this when you have a reference answer available.

```
from openai import AsyncOpenAI
from ragas.llms import llm_factory
from ragas.metrics.collections import ContextPrecision

# Setup LLM
client = AsyncOpenAI()
llm = llm_factory("gpt-4o-mini", client=client)

# Create metric
scorer = ContextPrecision(llm=llm)

# Evaluate
result = await scorer.ascore(
    user_input="Where is the Eiffel Tower located?",
    reference="The Eiffel Tower is located in Paris.",
    retrieved_contexts=[
        "The Eiffel Tower is located in Paris.",
        "The Brandenburg Gate is located in Berlin."
    ]
)
print(f"Context Precision Score: {result.value}")
```

Output:

```
Context Precision Score: 0.9999999999
```

 [latest](#) ▼

Synchronous Usage

If you prefer synchronous code, you can use the `.score()` method instead of `.ascore()`:

```
result = scorer.score(  
    user_input="Where is the Eiffel Tower located?",  
    reference="The Eiffel Tower is located in Paris.",  
    retrieved_contexts=[...]  
)
```

Context Utilization

The `ContextUtilization` metric evaluates whether retrieved contexts are useful by comparing each context against the generated response. Use this when you don't have a reference answer but have the response that was generated.

```
from openai import AsyncOpenAI  
from ragas.llms import llm_factory  
from ragas.metrics.collections import ContextUtilization  
  
# Setup LLM  
client = AsyncOpenAI()  
llm = llm_factory("gpt-4o-mini", client=client)  
  
# Create metric  
scorer = ContextUtilization(llm=llm)  
  
# Evaluate  
result = await scorer.ascore(  
    user_input="Where is the Eiffel Tower located?",  
    response="The Eiffel Tower is located in Paris.",  
    retrieved_contexts=[  
        "The Eiffel Tower is located in Paris.",
```

```
        "The Brandenburg Gate is located in Berlin."
    ]
)
print(f"Context Utilization Score: {result.value}")
```

Output:

```
Context Utilization Score: 0.9999999999
```

Note that even if an irrelevant chunk is present at the second position in the array, context precision remains the same. However, if this irrelevant chunk is placed at the first position, context precision reduces:

```
result = await scorer.ascore(
    user_input="Where is the Eiffel Tower located?",
    response="The Eiffel Tower is located in Paris.",
    retrieved_contexts=[
        "The Brandenburg Gate is located in Berlin.",
        "The Eiffel Tower is located in Paris."
    ]
)
print(f"Context Utilization Score: {result.value}")
```

Output:

```
Context Utilization Score: 0.49999999995
```

Legacy Metrics API

 [latest](#) ▼

The following examples use the legacy metrics API pattern. For new projects, we recommend using the collections-based API shown above.

 Deprecation Timeline

This API will be deprecated in version 0.4 and removed in version 1.0. Please migrate to the collections-based API shown above.

Example with SingleTurnSample

```
from ragas import SingleTurnSample
from ragas.metrics import LLMContextPrecisionWithoutReference

context_precision = LLMContextPrecisionWithoutReference(llm=evaluator_llm)

sample = SingleTurnSample(
    user_input="Where is the Eiffel Tower located?",
    response="The Eiffel Tower is located in Paris.",
    retrieved_contexts=["The Eiffel Tower is located in Paris."],
)

await context_precision.single_turn_ascore(sample)
```

Output:

```
0.9999999999
```

Context Precision without reference

The `LLMContextPrecisionWithoutReference` metric can be used without the availability of a reference answer. To estimate if are relevant, this method uses the LLM to compare each chunk in `retrieved_contexts` with the `response`.

 [latest](#) ▼

Example

```
from ragas import SingleTurnSample
from ragas.metrics import LLMContextPrecisionWithoutReference

context_precision = LLMContextPrecisionWithoutReference(llm=evaluator_llm)

sample = SingleTurnSample(
    user_input="Where is the Eiffel Tower located?",
    response="The Eiffel Tower is located in Paris.",
    retrieved_contexts=["The Eiffel Tower is located in Paris."],
)

await context_precision.single_turn_ascore(sample)
```

Output:

```
0.9999999999
```

Context Precision with reference

The `LLMContextPrecisionWithReference` metric can be used when you have both retrieved contexts and also a reference response associated with a `user_input`. To estimate if the retrieved contexts are relevant, this method uses the LLM to compare each chunk in `retrieved_contexts` with the `reference`.

Example

```
from ragas import SingleTurnSample
from ragas.metrics import LLMContextPrecisionWithReference

context_precision = LLMContextPrecisionWithReference(llm=evaluator_llm)

sample = SingleTurnSample(
    user_input="Where is the Eiffel Tower located?",
```

```
reference="The Eiffel Tower is located in Paris.",
retrieved_contexts=["The Eiffel Tower is located in Paris."],
)

await context_precision.single_turn_ascore(sample)
```

Output:

```
0.9999999999
```

Non LLM Based Context Precision

This metric uses non-LLM-based methods (such as [Levenshtein distance measure](#)) to determine whether a retrieved context is relevant.

Context Precision with reference contexts

The `NonLLMContextPrecisionWithReference` metric is designed for scenarios where both retrieved contexts and reference contexts are available for a `user_input`. To determine if a retrieved context is relevant, this method compares each retrieved context or chunk in `retrieved_contexts` with every context in `reference_contexts` using a non-LLM-based similarity measure.

Note that this metric would need the `rapidfuzz` package to be installed: `pip install rapidfuzz`.

Example

```
from ragas import SingleTurnSample
from ragas.metrics import NonLLMContextPrecisionWithReference

context_precision = NonLLMContextPrecisionWithReference()

sample = SingleTurnSample(
```

```
retrieved_contexts=["The Eiffel Tower is located in Paris."],  
reference_contexts=["Paris is the capital of France.", "The Eiffel Tower is one of the most famous landmarks in Paris."]  
)  
  
await context_precision.single_turn_ascore(sample)
```

Output:

```
0.9999999999
```

ID Based Context Precision

IDBasedContextPrecision provides a direct and efficient way to measure precision by comparing the IDs of retrieved contexts with reference context IDs. This metric is particularly useful when you have a unique ID system for your documents and want to evaluate retrieval performance without comparing the actual content.

The metric computes precision using `retrieved_context_ids` and `reference_context_ids`, with values ranging between 0 and 1. Higher values indicate better performance. It works with both string and integer IDs.

The formula for calculating ID-based context precision is as follows:

$$\text{ID-Based Context Precision} = \frac{\text{Number of retrieved context IDs found in reference context IDs}}{\text{Total number of retrieved context IDs}}$$

Example

```
from ragas import SingleTurnSample  
from ragas.metrics import IDBasedContextPrecision
```



```
sample = SingleTurnSample(  
    retrieved_context_ids=["doc_1", "doc_2", "doc_3", "doc_4"],  
    reference_context_ids=["doc_1", "doc_4", "doc_5", "doc_6"]  
)  
  
id_precision = IDBasedContextPrecision()  
await id_precision.single_turn_ascore(sample)
```

Output:

0.5

In this example, out of the 4 retrieved context IDs, only 2 ("doc_1" and "doc_4") are found in the reference context IDs, resulting in a precision score of 0.5 or 50%.

 December 11, 2025  December 11, 2025  GitHub      +6

  latest ▼